

- The frames per second (frame rate) that the program should maintain.

Finally, we tell the scene manager to start running by calling its `run()` method ⑤. The scene manager always maintains a single scene as the current scene—the one that the user sees and interacts with.

Notice that with this approach, the main program implements the initialization of a typical pygame program but does *not* build the main loop. Instead, the main loop is built into the scene manager itself.

Building the Scenes

To understand the interaction between the scene manager and any individual scene, I'll explain how a typical scene is built.

Each time through its loop, the scene manager calls a predefined set of methods in the current scene that are intended to handle events, do any per-frame actions, and draw anything that needs to be drawn in that scene. Therefore, the code of each scene must be split up into these methods. The approach makes use of polymorphism: each scene needs to implement a common set of methods.

Methods to Implement in Each Scene

Each scene is implemented as a class that inherits from the `Scene` base class defined in the *pyghelpers.py* file. Therefore, each scene must import *pyghelpers*. At a minimum, a scene needs to contain an `__init__()` method and must override the `getSceneKey()`, `handleInputs()`, and `draw()` methods from the base class.

Every scene must have a unique *scene key*—a string used by the scene manager to identify each scene. I recommend that you build a file with a name like *Constants.py* that contains the keys for all the scenes and import this file into each scene file. For example, the *Constants.py* file for the sample program contains:

```
# Scene keys (any unique values):
SCENE_A = 'scene A'
SCENE_B = 'scene B'
SCENE_C = 'scene C'
```
